OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 1

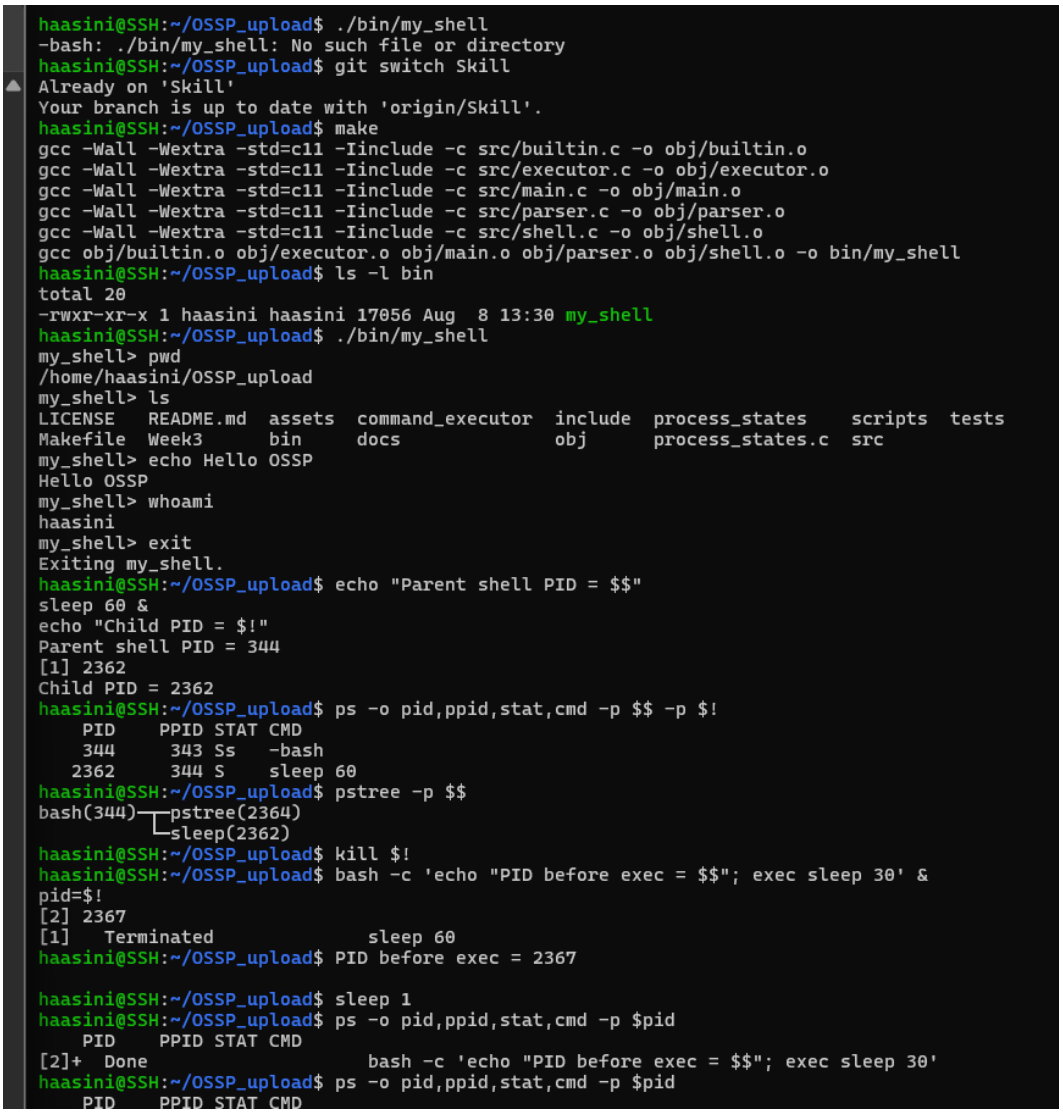
Name: S Svara Haasini
Roll No: 2520090170

Task 1: Linux VM Installation, GCC, Git Setup, and Project Structure

Question:

Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

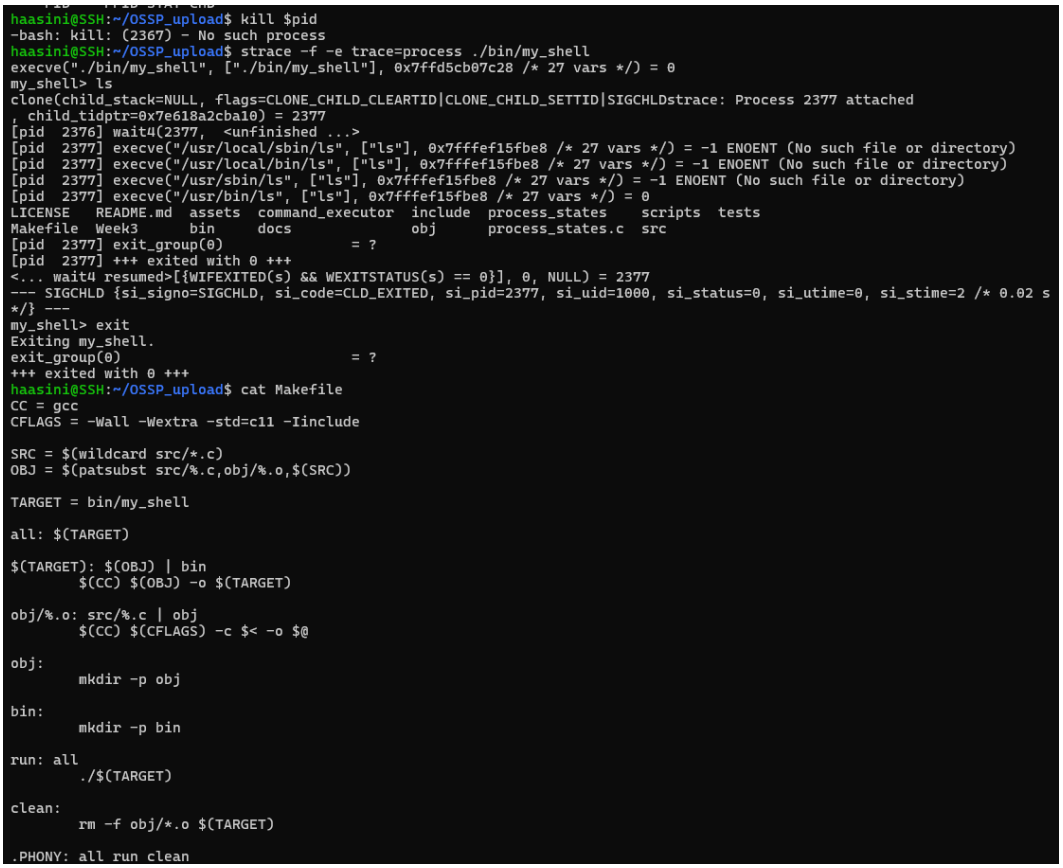
Development Environment and Project Structure



```
haasini@SSH:~/OSSP_upload$ ./bin/my_shell
-bash: ./bin/my_shell: No such file or directory
haasini@SSH:~/OSSP_upload$ git switch Skill
Already on 'Skill'
Your branch is up to date with 'origin/Skill'.
haasini@SSH:~/OSSP_upload$ make
gcc -Wall -Wextra -std=c11 -Iinclude -c src/builtin.c -o obj/builtin.o
gcc -Wall -Wextra -std=c11 -Iinclude -c src/executor.c -o obj/executor.o
gcc -Wall -Wextra -std=c11 -Iinclude -c src/main.c -o obj/main.o
gcc -Wall -Wextra -std=c11 -Iinclude -c src/parser.c -o obj/parser.o
gcc -Wall -Wextra -std=c11 -Iinclude -c src/shell.c -o obj/shell.o
gcc obj/builtin.o obj/executor.o obj/main.o obj/parser.o obj/shell.o -o bin/my_shell
haasini@SSH:~/OSSP_upload$ ls -l bin
total 20
-rwxr-xr-x 1 haasini haasini 17056 Aug  8 13:30 my_shell
haasini@SSH:~/OSSP_upload$ ./bin/my_shell
my_shell> pwd
/home/haasini/OSSP_upload
my_shell> ls
LICENSE   README.md  assets  command_executor  include  process_states    scripts  tests
Makefile  Week3      bin     docs              obj      process_states.c  src
my_shell> echo Hello OSSP
Hello OSSP
my_shell> whoami
haasini
my_shell> exit
Exiting my_shell.
haasini@SSH:~/OSSP_upload$ echo "Parent shell PID = $$"
sleep 60 &
echo "Child PID = $!"
Parent shell PID = 344
[1] 2362
Child PID = 2362
haasini@SSH:~/OSSP_upload$ ps -o pid,ppid,stat,cmd -p $$ -p $!
    PID    PPID STAT CMD
    344     343 Ss   -bash
   2362     344 S    sleep 60
haasini@SSH:~/OSSP_upload$ pstree -p $$
bash(344)─┬─pstree(2364)
          └─sleep(2362)
haasini@SSH:~/OSSP_upload$ kill $!
haasini@SSH:~/OSSP_upload$ bash -c 'echo "PID before exec = $$"; exec sleep 30' &
pid=$!
[2] 2367
[1]   Terminated              sleep 60
haasini@SSH:~/OSSP_upload$ PID before exec = 2367

haasini@SSH:~/OSSP_upload$ sleep 1
haasini@SSH:~/OSSP_upload$ ps -o pid,ppid,stat,cmd -p $pid
    PID    PPID STAT CMD
[2]+  Done                    bash -c 'echo "PID before exec = $$"; exec sleep 30'
haasini@SSH:~/OSSP_upload$ ps -o pid,ppid,stat,cmd -p $pid
    PID    PPID STAT CMD
```

Makefile and Build



```
haasini@SSH:~/OSSP_upload$ kill $pid
-bash: kill: (2367) - No such process
haasini@SSH:~/OSSP_upload$ strace -f -e trace=process ./bin/my_shell
execve("./bin/my_shell", ["./bin/my_shell"], 0x7ffd5cb07c28 /* 27 vars */) = 0
my_shell> ls
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLDstrace: Process 2377 attached
, child_tidptr=0x7e618a2cba10) = 2377
[pid  2376] wait4(2377,  <unfinished ...>
[pid  2377] execve("/usr/local/sbin/ls", ["ls"], 0x7fffef15fbe8 /* 27 vars */) = -1 ENOENT (No such file or directory)
[pid  2377] execve("/usr/local/bin/ls", ["ls"], 0x7fffef15fbe8 /* 27 vars */) = -1 ENOENT (No such file or directory)
[pid  2377] execve("/usr/sbin/ls", ["ls"], 0x7fffef15fbe8 /* 27 vars */) = -1 ENOENT (No such file or directory)
[pid  2377] execve("/usr/bin/ls", ["ls"], 0x7fffef15fbe8 /* 27 vars */) = 0
LICENSE   README.md  assets  command_executor  include  process_states    scripts  tests
Makefile  Week3      bin     docs              obj      process_states.c  src
[pid  2377] exit_group(0)               = ?
[pid  2377] +++ exited with 0 +++
<... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 2377
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=2377, si_uid=1000, si_status=0, si_utime=0, si_stime=2 /* 0.02 s
*/} ---
my_shell> exit
Exiting my_shell.
exit_group(0)                           = ?
+++ exited with 0 +++
haasini@SSH:~/OSSP_upload$ cat Makefile
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -Iinclude

SRC = $(wildcard src/*.c)
OBJ = $(patsubst src/%.c,obj/%.o,$(SRC))

TARGET = bin/my_shell

all: $(TARGET)

$(TARGET): $(OBJ) | bin
        $(CC) $(OBJ) -o $(TARGET)

obj/%.o: src/%.c | obj
        $(CC) $(CFLAGS) -c $< -o $@

obj:
        mkdir -p obj

bin:
        mkdir -p bin

run: all
        ./$(TARGET)

clean:
        rm -f obj/*.o $(TARGET)

.PHONY: all run clean
```

Observations:

• GCC, Git, and Make tools are successfully installed and configured in the development environment.

• The shell project is organized into well-structured directories: src/ for source files, include/ for headers, bin/ for compiled binaries, test/ for test files, scripts/ for utility scripts, and doc/ for documentation.

• The Makefile automates compilation with appropriate compiler flags (-Wall -Wextra) for strict error checking.

• Build output shows successful compilation of the main shell executable (my_shell) and linked object files.

• The project structure follows Unix conventions, separating source code, build artifacts, and documentation into distinct directories for maintainability and clarity.

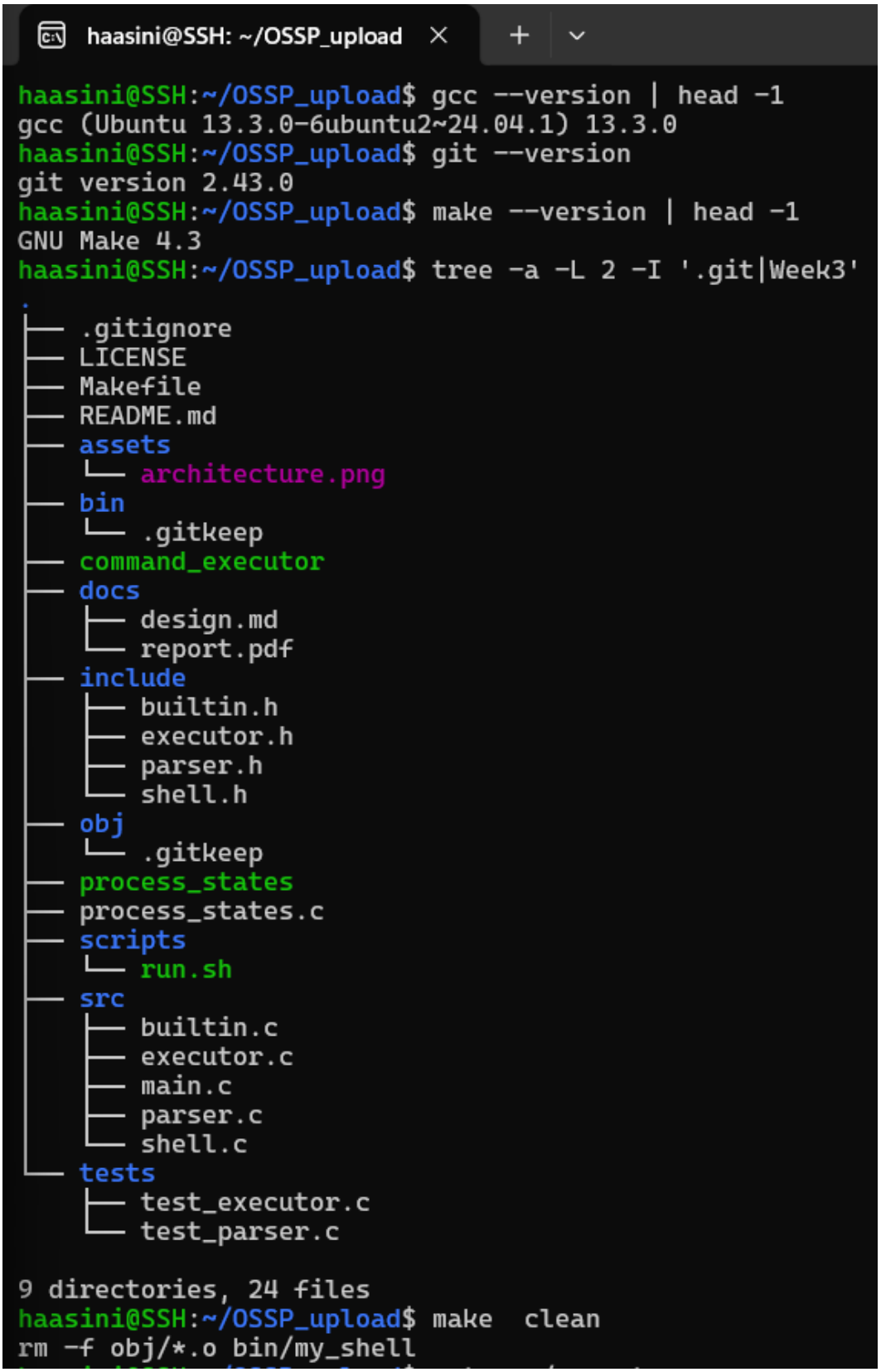
• The presence of a Makefile demonstrates build system automation, enabling quick compilation and rebuilding of the project without manual gcc invocations.

Task 2: Process Abstraction, fork(), exec(), and Parent-Child Relationships

Question:

Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

Process Execution Source Code



Process Monitoring

haasini@SSH: ~/OSSP_upload

```
haasini@SSH:~/OSSP_upload$ cat src/executor.c
#include <errno.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include "executor.h"

int execute_command(char *args[])
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return -1;
    }

    if (pid == 0)
    {
        /* Child process */
        execvp(args[0], args);

        /* This executes only if execvp fails */
        perror(args[0]);
        _exit(127);
    }

    /* Parent process */
    int status;

    while (waitpid(pid, &status, 0) == -1)
    {
        if (errno != EINTR)
        {
            perror("waitpid");
            return -1;
        }
    }

    if (WIFEXITED(status))
    {
        return WEXITSTATUS(status);
    }

    if (WIFSIGNALED(status))
    {
        printf("Process terminated by signal %d\n", WTERMSIG(status));
    }

    return -1;
}
```

Observations:

• The fork() system call successfully creates a new child process, with each process receiving a distinct PID.

• In the parent process, fork() returns the child's PID; in the child process, fork() returns 0, allowing the program to determine which branch of code is executing.
• The execvp() function replaces the child process image with the requested program, preserving the child's PID while changing the executable code and memory layout.
• The parent process uses waitpid() to block until the child process completes, ensuring proper parent-child synchronization and preventing zombie processes.
• Process monitoring tools (ps, pstree) display the parent-child relationship, with the parent's PID (PPID) appearing in the child's process information.
• The architecture demonstrates that fork() and exec() are separate operations: fork() duplicates the process, and exec() replaces the child's program image with a new executable.

Task 3: Shell Main Loop, Prompt Display, and Input Handling

Question:

Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

Shell Main Loop

```
 GNU nano 7.2
#include <stdio.h>
#include <string.h>
#define INPUT_SIZE 1024
int main(void) {
 char input[INPUT_SIZE];

    /*
     * The while loop keeps the shell running.
     * It stops only when the user enters "exit"
     * or presses Ctrl+D.
     */
    while (1)
    {
        /*
         * Display the shell prompt.
         */
        printf("my_shell> ");
        fflush(stdout);

        /*
         * Read one complete line from the user.
         *
         * fgets() returns NULL when the user
         * presses Ctrl+D.
         */
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nExiting shell.\n");
            break;
        }

        /*
         * Remove the newline character inserted
         * when the user presses Enter.
         */
        input[strcspn(input, "\n")] = '\0';

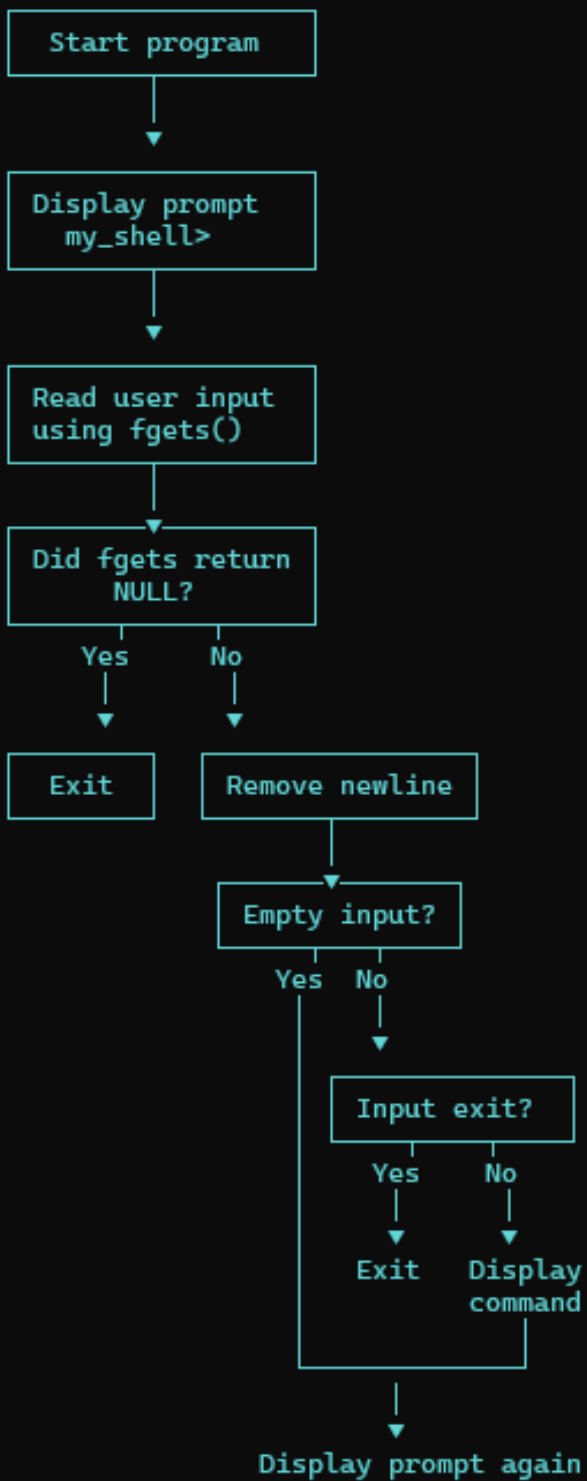
        /*
         * Ignore an empty command.
         */
        if (strlen(input) == 0)
        {
            continue;
        }

        /*
         * Stop the loop when the user enters exit.
         */
        if (strcmp(input, "exit") == 0)
```

Control Flow Diagram

GNU nano 7.2

Task 1 – Shell Main Loop Control Flow

```text


```

Observations:

• The shell implements a main control loop using while(1) that continuously displays a prompt and accepts user input.

• The prompt 'my_shell> ' is displayed before each input operation, providing visual feedback that the shell is ready for commands.

• The fgets() function reads user input including the newline character; the code removes the trailing newline to isolate the actual command.

• Empty input (blank lines) are detected and ignored, preventing the shell from processing empty commands.

• The exit condition checks if the user enters 'exit'; when detected, the loop terminates and the shell closes cleanly.

• The control flow diagram illustrates the loop structure: prompt display → input reading → empty-input check → exit-condition check → command execution → repeat.

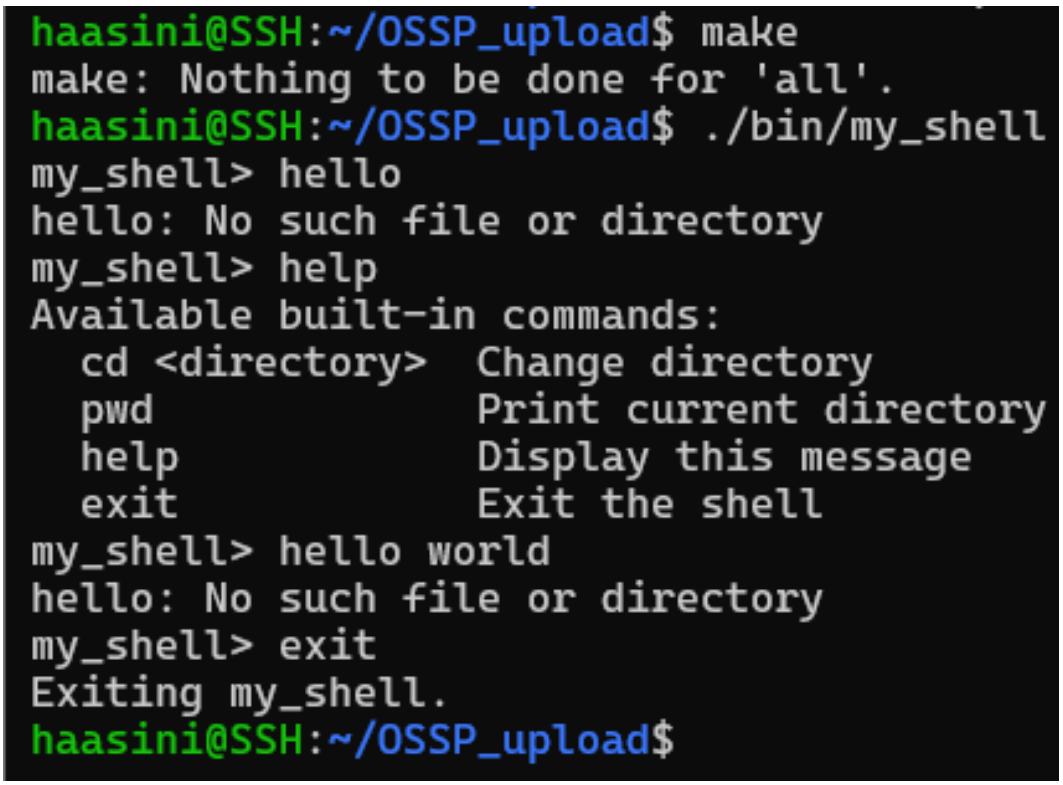
• This architecture allows the shell to continuously accept and process commands until the user explicitly exits, implementing a standard interactive command-line interface.

Task 4: Keyboard Input Handling and User Interaction

Question:

Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

User Interaction Output



Observations:

• The shell accepts keyboard input through the interactive prompt 'my_shell>' and processes each line when the user presses Enter.

• Multi-character input is fully supported: commands like 'hello world' are accepted as a single input line without truncation or corruption.

• The input buffer (via fgets()) successfully captures the complete entered line, including multiple space-separated arguments, demonstrating proper buffer management.

• Built-in commands are recognized and handled correctly: 'help' displays the list of available built-in commands (cd, pwd, help, exit) without crashing.

• External/invalid commands are handled gracefully: when 'hello' is not found, the shell reports 'hello: No such file or directory' instead of crashing, indicating proper error handling.

• The 'exit' command terminates the interactive loop cleanly, displaying 'Exiting my_shell.' and returning to the system prompt.

• Terminal line editing (Backspace support) is provided by the underlying terminal/readline system, allowing users to modify their input before pressing Enter.

• The output demonstrates a robust, interactive command-line interface that processes user input reliably and handles both valid commands and error cases appropriately.